

FOLIOPRESS

Grupo 10

AUTHOR	Gabriel
DATE	13/06/2026

Contents

Corrección del proyecto

Puntaje final

Resumen general

Puntaje por categoría

Fortalezas

Faltas o aspectos críticos a revisar

Recomendaciones de mejora

Observaciones sobre el proceso de trabajo

Observaciones sobre el uso de IA

Corrección del proyecto

Puntaje final

Nota: 96/100

Resumen general

Mini Finance es un proyecto sólido, muy por encima de la media. Presenta cinco páginas HTML reales, un sistema de autenticación completo, modularidad estricta en JavaScript (módulos ES + scripts clásicos separados por responsabilidad), uso intensivo y bien justificado de `localStorage` para persistir datos financieros, accesibilidad básica bien resuelta y un sistema de diseño con tokens, modo claro/oscuro, modo dislexia y modo daltónico. La estética está cuidada, el responsive es real y el deploy funciona. La principal observación pendiente es que el **desafío obligatorio de la idea Mini Finance no está cumplido en su totalidad**: la alerta visual de "no se alcanza la meta de ahorro" sí se renderiza en el DOM, pero la alerta de "superar un límite de gasto" no existe como concepto en el proyecto, y la representación del balance negativo es un cambio de clase en una card, no una alerta dentro del contenedor de alertas. También se detecta que la fuente OpenDyslexic referenciada por CDN no carga en producción por una política de seguridad del navegador (ORB), por lo que el "modo dislexia" termina cayendo a la fuente del sistema.

Puntaje por categoría

Categoría	Puntaje obtenido	Puntaje máximo	Comentario breve
HTML y estructura	15	15	Cinco HTML navegables, semántica completa (header/nav/main/section/article/aside/footer), rutas relativas correctas, formularios bien etiquetados.
CSS, diseño visual y responsive	20	20	CSS propio, sistema de tokens, 27 media queries, dark/light/daltonismo/dislexia, Flex/Grid bien usado.
JavaScript e interactividad	22	25	Funcionalidades reales de sobra (>10), DOM bien trabajado y código modular, pero el desafío obligatorio de Mini Finance está resuelto solo a medias.
Storage y persistencia	10	10	<code>localStorage</code> para movimientos, metas, sesión, accesibilidad, logros, footer y cookies. Datos persistidos y recuperados correctamente.
Git, GitHub y proceso	9	10	Historial claro, 158 commits, ramas <code>main</code> y <code>develop</code> con PRs (#1, #2, #22, #23), Conventional Commits bien usados con detalle menor en mayúsculas iniciales.
Deploy y funcionamiento	5	5	Deploy accesible, redirige correctamente, recursos cargan (la única excepción es la fuente OpenDyslexic bloqueada por el navegador).
README e informe de IA	10	10	README completo y bien organizado; informe de IA extenso, honesto y específico, contesta las 10 preguntas de la consigna.
Accesibilidad básica	5	5	<code>alt</code> en imágenes, <code><label></code> en todos los inputs, skip link, foco visible, ARIA en componentes interactivos, contraste correcto y estados no dependientes solo del color.
Total	96	100	

Fortalezas

- **Modularidad real:** la separación en `config.js`, `state.js`, `dom.js`, `render.js`, `events.js`, `main.js` y módulos ES por página (`simulator.js`, `resumen.js`, `market-rates.js`) hace que cada archivo tenga una responsabilidad clara. El refactor que extrajo `labels.js` para evitar el `SyntaxError` por `export` en `state.js` muestra criterio técnico.
- **Sistema de diseño robusto:** 14 archivos CSS coordinados por `tokens.css`, con variables para light/dark, escalado de tipografía, estados semánticos (`--danger-*`, `--warning-*`, `--success-*`) y un override limpio (`body.dark-mode`) en lugar de duplicar reglas.
- **Accesibilidad notable:** skip link, foco visible, `aria-expanded` / `aria-controls` / `aria-pressed` / `aria-haspopup` / `aria-live`, `<label for>` en todos los formularios, `aria-current="page"` en el link activo del nav, y `role="dialog"` en Pesito. Además de los modos dislexia y daltónico persistentes.
- **Funcionalidades reales integradas:** autenticación completa con tres vistas y validaciones, registro de ingresos/gastos con categorías, meta de ahorro con slider de doble escala (monto / porcentaje), 12 logros desbloqueables, cotizaciones de cripto en vivo (CoinGecko) y conversor de divisas (exchangerate-api), chat asistente Pesito con árbol de decisiones, banner de cookies y footer desplegable persistente.
- **Persistencia significativa:** los movimientos, metas, sesión, modo oscuro, modo dislexia, modo daltónico, logros, estado del footer y aceptación de cookies viven en `localStorage` y se restauran al recargar.
- **Historial de Git claro y trabajoso:** 158 commits, 4 autores con participación verificable (Alejo 68, Francisco 46, Karen 22, Alejo M 22), múltiples ramas de feature y merges, PRs desde `develop` hacia `main`.

Faltas o aspectos críticos a revisar

- **Desafío obligatorio de Mini Finance incompleto.** La consigna exige "alertas visuales en el DOM cuando el usuario supere un límite de gasto o no alcance una meta de ahorro". El proyecto implementa solo la segunda parte: en `state.js`, `checkAlerts()` empuja un `alert--warning` al `#alerts-container` cuando el balance no alcanza la meta. La primera parte no existe: no hay un "límite de gasto" configurable por el usuario y, por lo tanto, no hay alerta cuando se lo supera. La representación del balance negativo existe, pero es un cambio de clase (`is-negative`) en la card de saldo, no una alerta dentro del contenedor `#alerts-container` (que es el patrón ya establecido en el resto del proyecto).

- **Fuente OpenDyslexic bloqueada en el deploy.** El `<link>` apunta a `https://cdn.jsdelivr.net/npm/opendyslexic@2.0.0/opendyslexic.min.css`, que el navegador bloquea por política ORB. El modo dislexia cae a la fuente del sistema. La accesibilidad no se rompe, pero el modo pierde su propósito real.
- **Conventional Commits con algunos desvíos.** Varios commits usan mayúscula inicial en el prefijo (`Feat:` , `Fix:` , `Add`) en lugar de la convención lowercase (`feat:` , `fix:`). Es un detalle, pero rompe el parseo automático por herramientas.
- **Balance siempre arranca como positivo en el HTML.** En `index.html` la card de saldo tiene la clase `is-positive` hardcodeda. La lógica en `render.js` la corrige en tiempo de ejecución, pero deja un breve parpadeo al cargar.
- **El inline style en los logos** (`style="width: 48px; height: 48px; margin-right: 0.5rem; ..."`) es menor, pero rompe la separación de responsabilidades: lo correcto sería una clase en CSS.

Recomendaciones de mejora

- Implementar el "límite de gasto" como input configurable (por categoría o global) y añadir un `alert--danger` al `#alerts-container` cuando los gastos del mes superen ese umbral, siguiendo el mismo patrón que ya usan para la meta de ahorro.
- Resolver el bloqueo de OpenDyslexic: descargar la fuente y servirla localmente desde `assets/` , o cambiarla por una alternativa auto-hospedada.
- Estandarizar los prefijos de commit a lowercase (`feat:` , `fix:` , `style:` , `docs:` , `refactor:`) y agregar un `commitlint` / hook para evitar regresiones.
- Quitar los `style=""` inline de los logos y mover las dimensiones a clases CSS (`header__logo` , `footer__brand-logo-img`).
- Evaluar mover la clase inicial del balance a un estado neutro (`is-loading`) en lugar de `is-positive` , para evitar el salto visual antes del primer render del JS.

Observaciones sobre el proceso de trabajo

El historial de Git es de los más claros del curso. Se observa una rama `main` y una `develop` con flujo real, ramas por feature (`feat/pesito-chatbot-and-design-lab` , `feat/amplify-resumen` , `feat/dashboard-metrics-and-simulator-tweaks` , `feature/cotizaciones-dashboard` , `docs/informe-ia` , etc.) y PRs documentados (#1, #2, #22, #23). Los 158 commits son progresivos y describen cambios concretos: agregar el slider de ahorro, ajustar el color del footer, refactorizar `resumen.js` para evitar `innerHTML` , aislar

tokens de tematización, etc. Los Conventional Commits están bien usados en su mayoría (`feat:` , `fix:` , `style:` , `refactor:` , `docs:` , `chore:`); las excepciones son los prefijos con mayúscula (`Feat:` , `Fix:` , `Add` , `Update` , `Refine`) que conviene normalizar. La distribución de trabajo entre los tres integrantes (Alejo Martínez, Francisco Chiressi, Karen Ledesma) es verificable y coherente con el reparto de páginas declarado en el informe de IA.

Observaciones sobre el uso de IA

El informe de uso de IA es uno de los más completos y honestos de la cohorte. Contesta las diez preguntas de la consigna con ejemplos concretos: herramientas usadas (OpenCode con Gemini 3.1 Pro y Kimi 2.6, Claude Opus 4.8, Claude Sonnet 4.6, `skills frontend-design` , `interface-design` , `mobile-first-design` y `buenas-practica-mini-finance`), prompts textuales que sirvieron y otros que hubo que corregir, errores de la IA que fueron detectados y resueltos manualmente (pérdida de `accessibility.css` y `login.js` en una sesión, HTML corrupto al insertar el menú de usuario, commit no autorizado, `replaceChildren()` mal escrito como `replaceChild()` , `state.js` con `export` cargado como classic script, BEM con `___` incompatible con la convención del proyecto). El equipo demuestra entender el código asistido: explica por qué descartó la solución drástica de convertir todos los scripts a módulos, por qué eligió `body: not ( .dark-mode)` para los tokens light-only, y por qué descartó la API de Frankfurter. La sección de "qué puede explicar cada integrante" deja claro que hubo apropiación real del trabajo.